



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

**A PROJECT REPORT ON
THESIS & PROJECT MANAGEMENT SYSTEM
(TPMS)**

Submitted By:

Prabesh BC (PUL080BCT054)
Shreeyut Thapa (PUL080BCT080)
Subesh Yadav (PUL080BCT084)

Submitted To:

Department of Electronics &
Computer Engineering

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled “**Thesis & Project Management System (TPMS)**” submitted by **Prabesh BC (PUL080BCT054)**, **Shreeyut Thapa (PUL080BCT080)**, and **Subesh Yadav (PUL080BCT084)** in partial fulfillment of the requirements for the Bachelor of Engineering degree in Computer Engineering.

.....

Supervisor

Er. Bishnu Tamang

Assistant Professor

Department of Electronics and Computer

Engineering,

Pulchowk Campus, IOE, TU.

.....

External Examiner

Dr. Kiran Mainali

Associate Professor

Department of Electronics and Computer

Engineering,

Pulchowk Campus, IOE, TU.

Date of approval: August 12, 2026

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying, publication, or other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
Pulchowk Campus, Institute of Engineering, TU
Lalitpur, Nepal.

Acknowledgements

We express our deepest gratitude to our project supervisor, **Er. Bishnu Tamang**, Assistant Professor at the Department of Electronics and Computer Engineering, Pulchowk Campus, for his unwavering guidance, consistent encouragement, and expert feedback throughout every phase of this project. His deep knowledge of software engineering practices and willingness to invest time in our queries shaped both the technical robustness and academic quality of this work.

We extend our sincere thanks to **Prof. Dr. Sashidhar Ram Joshi**, Dean of the Institute of Engineering, and the faculty members of the Department of Electronics and Computer Engineering for providing the necessary infrastructure, laboratory facilities, and a conducive learning environment. The structured academic framework provided by the department served as the direct inspiration for the domain problem this system addresses.

Special acknowledgement is due to the program coordinators and faculty members who graciously shared their experiences with the existing paper-based evaluation workflows, providing invaluable ground-truth information during the requirements elicitation phase. Their insights into operational pain points directly shaped the feature set of TPMS.

Finally, we express heartfelt appreciation to our fellow classmates, peers in BCT 080 batch, and our families for their constant moral support, patience, and encouragement throughout the development of the Thesis & Project Management System.

Abstract

The management of undergraduate projects and postgraduate theses at Pulchowk Campus, Institute of Engineering (IOE), has traditionally relied on fragmented manual workflows, paper evaluation sheets, notice board communications, and uncoordinated email exchanges. This manual paradigm leads to supervision tracking opacity, severe administrative bottlenecks during evaluation periods, error-prone mark aggregation, and absence of a centralized document repository. To address these systemic inefficiencies, we present the **Thesis & Project Management System (TPMS)**, a comprehensive, secure, multi-tier web platform tailored specifically to the academic governance structures of the Department of Electronics and Computer Engineering (DOECE), Pulchowk Campus.

TPMS integrates five distinct role-based access levels—Student, Supervisor, External Examiner, Program Coordinator, and Maintainer—across six academic degree programs (BCT, BEI, MSNCS, MSICE, MSDSA, MSCSKE). Built upon a modern React 18 single-page application frontend and a Node.js/Express REST API backend connected to a PostgreSQL 16 database via Prisma ORM, TPMS encapsulates the complete project lifecycle: group formation, bulk Excel student importing with anomaly detection, supervisor assignment, document submission, and criteria-based rubric evaluation.

Crucially, TPMS features a server-side **Puppeteer PDF rendering engine** that dynamically compiles official, university-branded A4 evaluation sheets with automatic numerical-to-word mark conversion, as well as a **Retrieval-Augmented Generation (RAG) AI Chatbot microservice** (built with FastAPI, ChromaDB, and Llama-3/Gemini) that offers real-time proposal analysis and guidelines-based Q&A. Empirical validation across 100+ test iterations demonstrates 100% mark calculation accuracy, PDF generation under 1.2 seconds, and seamless operation across multi-batch academic rosters.

Keywords: *Academic Management, Thesis Governance, Role-Based Access Control, Headless PDF Rendering, Retrieval-Augmented Generation, React, Node.js, Prisma ORM, PostgreSQL*

Contents

Page of Approval	ii
Copyright	iii
Acknowledgements	iv
Abstract	v
List of Figures	ix
List of Tables	x
List of Abbreviations	xii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope & Limitations	3
1.4.1 Scope	3
1.4.2 Limitations	4
1.5 Report Organization	4
2 Literature Review	5
2.1 Related Work	5
2.1.1 Learning Management Systems and Their Limitations	5
2.1.2 Thesis Repository Systems	5
2.1.3 Academic Project Management Tools in Nepali Context	5
2.2 Related Theory	6
2.2.1 Single-Page Application Architecture	6
2.2.2 RESTful API Design and JWT Authentication	6
2.2.3 Object-Relational Mapping with Prisma	6
2.2.4 Server-Side Headless PDF Generation	6

2.2.5	Retrieval-Augmented Generation (RAG)	7
2.2.6	Agile Software Development	7
3	Methodology	8
3.1	Software Development Life Cycle Model	8
3.2	Requirement Analysis & Feasibility Study	8
3.2.1	Functional Requirements	8
3.2.2	Non-Functional Requirements	9
3.2.3	Feasibility Study	9
3.3	Sprint Plan	10
3.4	Technology Stack Selection	10
3.5	Security Design & RBAC Policy	11
4	System Design	13
4.1	System Architecture	13
4.1.1	Presentation Tier (Client Layer)	13
4.1.2	Application Tier (Backend Processing Layer)	14
4.1.3	Data Tier (Persistence Layer)	14
4.1.4	Inter-Tier Communication	15
4.2	Use Case Diagram	15
4.3	Entity-Relationship Diagram	15
4.4	Data Flow Diagram (Level 1)	16
4.5	Behavioral Diagrams	17
4.5.1	Excel Batch Import Sequence	17
4.5.2	Supervisor Assignment Sequence	18
4.5.3	Evaluation & PDF Generation Sequence	19
4.6	Project Schedule	19
5	Experimental Setup & Implementation	20
5.1	Development Environment	20
5.2	Database Schema Implementation	20
5.3	PDF Rendering Engine Implementation	21
5.4	RAG AI Microservice Setup	22
5.5	Excel Bulk Import Module	23
6	Results & Discussion	24
6.1	System Deployment & User Roles	24

6.2	PDF Evaluation Sheet Generation Performance	24
6.3	Evaluation Mark Accuracy Testing	24
6.4	RBAC Security Enforcement Testing	25
6.5	Excel Import Anomaly Detection Accuracy	25
6.6	RAG Chatbot Evaluation	25
6.7	Comparative Analysis	26
7	Conclusions	27
8	Limitations and Future Enhancement	28
8.1	Current Limitations	28
8.2	Future Enhancement Roadmap	28
	References	29
A	Database Schema Source (Prisma Excerpt)	32
B	API Endpoint Reference	34

List of Figures

4.1	TPMS 3-Tier System Architecture — Presentation, Application, and Data Layers	13
4.2	TPMS Use Case Diagram — Actor-System Interactions	15
4.3	Entity-Relationship Diagram — TPMS Core Database Schema with Data Types	16
4.4	Data Flow Diagram (Level 1, Yourdon-Coad Notation)	17
4.5	Sequence Diagram: Excel Bulk Roster Import & Registration Workflow . . .	18
4.6	Sequence Diagram: Supervisor Assignment & Non-Blocking Email Dispatch	18
4.7	Sequence Diagram: Marks Entry, Calculation & Puppeteer PDF Generation	19
4.8	Project Development Gantt Chart (16-Week Engineering Lifecycle)	19

List of Tables

- 3.1 Functional Requirements of TPMS 8
- 3.2 Technology Stack and Selection Rationale 11
- 5.1 System Environment Configuration 20
- 6.1 Puppeteer PDF Generation Performance Benchmarks (n=100 per type) . . . 24
- 6.2 RBAC Security Enforcement Test Results 25
- 6.3 RAG Chatbot Response Quality Evaluation (n=30 questions) 26
- 6.4 Operational Comparison: Legacy Manual Process vs. TPMS Platform 26
- B.1 TPMS REST API Endpoint Summary 34

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BCT	Bachelor in Computer Engineering
BEI	Bachelor in Electronics, Communication & Information Engineering
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DOECE	Department of Electronics and Computer Engineering
ER	Entity-Relationship
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IOE	Institute of Engineering
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
ORM	Object-Relational Mapping
PDF	Portable Document Format
PKI	Public Key Infrastructure
RBAC	Role-Based Access Control
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SPA	Single Page Application
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TLS	Transport Layer Security
TPMS	Thesis & Project Management System
TU	Tribhuvan University
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XLSX	Excel Open XML Spreadsheet Format

1. Introduction

1.1 Background

Academic project work and thesis research represent the pinnacle of undergraduate and postgraduate engineering education at Pulchowk Campus, Institute of Engineering (IOE), Tribhuvan University. For undergraduate programs such as Bachelor in Computer Engineering (BCT) and Bachelor in Electronics, Communication & Information Engineering (BEI), students complete both a *Minor Project* in their third academic year and a *Major Project* in their fourth and final year. Concurrently, postgraduate students across Master of Science programs—including MSNCS, MSICE, MSDSA, and MSCSKE—undertake extensive thesis research spanning a proposal defense, one or more mid-term progress evaluations, and a final viva-voce defense.

The combined student population engaged in such academic project activities at DOECE exceeds several hundred individuals per academic year, supervised by a relatively limited pool of faculty. The organizational complexity of mapping student groups to available supervisors, scheduling defense examination panels, distributing evaluation rubrics to both internal supervisors and external examiners, and ultimately consolidating marks for forwarding to the university examination section is substantial.

Despite the high technical caliber of research produced by IOE students, the administrative orchestration of these academic activities has historically relied on physical paper evaluation sheets, manual group registration forms, notice board announcements for defense schedules, and uncoordinated email exchanges between students, supervisors, and coordinators. Departmental program coordinators face significant logistical overhead when managing hundreds of enrolled students, verifying roll number assignments, mapping project groups to eligible supervisors, distributing evaluation rubrics to external examiners, and compiling physical marks into final grade sheets [1].

This operational context presents a clear need for a purpose-built digital platform that addresses the unique workflow requirements of DOECE’s academic project and thesis administration—one that understands the specific role hierarchy, degree program differentiation, and evaluation sheet format mandated by Tribhuvan University.

1.2 Problem Statement

The legacy paper-based project and thesis management workflow at Pulchowk Campus exhibits several critical operational drawbacks:

1. **High Administrative Burden:** Program coordinators manually track student groups, verify roll number validity against printed class rosters, validate project title uniqueness, and physically match student groups with available faculty supervisors using printed lists maintained across academic years and degree programs.
2. **Supervision & Progress Opacity:** Students lack real-time visibility into their supervisor assignment status. Supervisors cannot efficiently track which groups are under their supervision. Department heads cannot generate consolidated overview reports without manual data aggregation.
3. **Error-Prone Evaluation Compilation:** Supervisors and external examiners manually write numerical marks across five evaluation criteria, calculate totals by hand, convert total scores into written words (e.g., *“Eighty-Seven Point Five Only”*), and submit physical evaluation sheets. Manual calculations frequently introduce transcription errors, arithmetic mistakes, and inconsistent word conversion formats, causing delays in forwarding results to the examination section.
4. **Document Management Disarray:** Project proposals, progress reports, and thesis manuscripts are exchanged via scattered email attachments without a centralized repository, resulting in loss of version history, lack of automated proposal review tools, and no audit trail for submission timestamps.
5. **No AI-Assisted Feedback:** Students preparing proposals currently have no automated tool to verify alignment with departmental guidelines, assess methodology completeness, or receive instant feedback on formatting compliance—instead relying entirely on manual supervisor review.

1.3 Objectives

The primary objective of this project is to design, develop, test, and deploy a secure, web-based Thesis & Project Management System (TPMS) customized for Pulchowk Campus, IOE, DOECE. The specific sub-objectives are:

- To implement a granular five-role security model (Student, Supervisor, External Examiner, Program Coordinator, Maintainer) with role-specific navigation and program-scoped data isolation.

- To build an automated Excel batch import module capable of parsing DOECE class rosters, detecting data anomalies including duplicate roll numbers and missing supervisors, and auto-provisioning student groups with supervisor assignments.
- To develop an interactive evaluation workbench with criteria-based mark entry, real-time total score computation, automatic numerical-to-word mark transformation, and submission validation.
- To integrate a server-side Puppeteer PDF rendering engine that generates official, university-formatted A4 evaluation sheets including institutional headers, logo, supervisor and examiner signatures, ready for digital distribution or physical printing.
- To implement a Retrieval-Augmented Generation (RAG) AI Chatbot microservice to assist students with proposal guideline queries, document compliance feedback, and formatting suggestions.
- To support all six DOECE degree programs (BCT, BEI, MSNCS, MSICE, MSDSA, MSCSKE) within a single, multi-tenant platform instance with program-scoped data partitioning.

1.4 Scope & Limitations

1.4.1 Scope

The scope of TPMS encompasses the complete academic administration lifecycle for Minor and Major projects for BCT and BEI undergraduate programs, as well as Master thesis workflows for MSNCS, MSICE, MSDSA, and MSCSKE programs within the DOECE department. The system covers:

- Student self-service: group formation, invitation acceptance, proposal submission, and document upload.
- Faculty services: supervisor acceptance, recommendation letters, evaluation mark entry.
- Coordinator services: batch student import, supervisor assignment, exam schedule publication, mark forwarding.
- Administrative services: program and batch management, user account provisioning.
- AI services: proposal document analysis, guideline QA chatbot.

1.4.2 Limitations

- The current release is tailored to DOECE-specific evaluation formats and requires schema adaptation for deployment in other engineering departments at Pulchowk Campus.
- Digital signature integration uses PDF approval tokens rather than PKI hardware tokens; physical signature on printed sheets remains the formal approval mechanism.
- The RAG AI service response quality is bounded by the quality and coverage of ingested DOECE guideline documents.
- Offline operation is not supported; active network connectivity to the central server is required.

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 presents a review of related work and the theoretical foundations underlying the system. Chapter 3 describes the methodology adopted for system development. Chapter 4 provides detailed system design including architecture, data models, and behavioral diagrams. Chapter 5 covers the experimental setup and implementation details. Chapter 6 presents results and discussion. Chapter 7 draws conclusions, and Chapter 8 discusses limitations and future enhancement opportunities.

2. Literature Review

2.1 Related Work

2.1.1 Learning Management Systems and Their Limitations

Digital transformation in higher education has driven widespread adoption of Learning Management Systems (LMS) such as Moodle [2], Canvas [3], and Blackboard. These platforms are primarily designed for content delivery, quiz management, and course-level gradebooks. While they support assignment submission, they do not model the complex workflow of academic project administration—which involves multi-student group formation with invitation-based confirmation, multiple external examiner roles, multi-criterion rubric evaluation, and institutional document generation.

2.1.2 Thesis Repository Systems

Open-source institutional repository platforms such as DSpace [4] and ePrints provide structured digital archiving of completed theses and dissertations. However, they function as post-completion archives rather than process management tools. They offer no workflow support for group formation, supervisor negotiation, mark entry, or live PDF sheet compilation.

2.1.3 Academic Project Management Tools in Nepali Context

Prior studies in the Nepali engineering education context have highlighted the necessity of domain-specific academic workflow automation [5]. Existing departmental information systems at other campuses are typically built as basic CRUD applications with limited workflow orchestration and no automated evaluation sheet generation.

To the authors’ knowledge, no prior system developed for Pulchowk Campus or IOE supports the specific combination of: multi-program role-based access, automated Excel roster import, Puppeteer-based TU-formatted PDF evaluation sheet generation, and integrated AI proposal analysis—all in a single coherent platform.

2.2 Related Theory

2.2.1 Single-Page Application Architecture

Modern web applications leverage SPA patterns where the web browser loads a single HTML shell and dynamically updates the DOM via asynchronous REST API calls without full page reloads. React 18, utilizing a virtual DOM and concurrent rendering, enables highly reactive and composable UI components. The component-based architecture of React facilitates reusable role-specific dashboard views, while React Router v6 provides client-side navigation with URL-based route guards [6].

2.2.2 RESTful API Design and JWT Authentication

Representational State Transfer (REST) provides a scalable, stateless client-server architectural style [7]. TPMS implements REST endpoints following standard HTTP verb semantics (GET, POST, PUT, PATCH, DELETE) with JSON request and response bodies. Authentication in stateless REST APIs is achieved via JSON Web Tokens (JWT), which consist of a base64-encoded header specifying the signing algorithm, a payload containing user ID, role, program scope, and expiry timestamp, and a cryptographic signature computed using HMAC-SHA256.

2.2.3 Object-Relational Mapping with Prisma

Prisma 5 is a next-generation ORM for Node.js that replaces raw SQL with a type-safe declarative schema. The `schema.prisma` file serves as the single source of truth for the relational data model, from which Prisma auto-generates TypeScript client bindings. Prisma Migrate produces versioned SQL migration scripts, ensuring that schema evolution is reproducible and reversible. Prisma Client provides eager loading via the `include` directive, reducing N+1 query problems common in naive ORM usage [8].

2.2.4 Server-Side Headless PDF Generation

Generating dynamic HTML templates into print-ready A4 PDF documents requires headless browser rendering. Puppeteer controls a headless Chromium instance via the Chrome DevTools Protocol, executing pixel-perfect CSS Paged Media layout algorithms (`@page { size: A4; margin: 15mm; }`) to render institutional headers, logos, signature lines, and mark grids. This approach guarantees visual fidelity identical to browser-rendered pages, without requiring a physical display [9].

2.2.5 Retrieval-Augmented Generation (RAG)

Large Language Models (LLMs) can suffer from factual hallucination when queried about specific institutional guidelines they were not trained on. Retrieval-Augmented Generation (RAG) [10] mitigates this by: (1) embedding domain documents into a dense vector space; (2) at query time, performing semantic similarity search to retrieve the top- k most relevant text chunks; and (3) prepending these chunks into the LLM context prompt to generate verified, grounded answers. ChromaDB provides the persistent vector store with cosine similarity indexing.

2.2.6 Agile Software Development

The system was developed following Scrum-based Agile methodology [11]. Agile’s iterative sprint model was particularly suitable given the evolving nature of user requirements discovered through coordinator interviews. Short feedback cycles allowed rapid incorporation of DOECE-specific constraints—such as evaluation stage naming conventions and program-level data scoping—into the codebase.

3. Methodology

3.1 Software Development Life Cycle Model

TPMS was developed following an Agile Scrum methodology over a 16-week timeline, divided into four 4-week sprints with a one-week buffer. The Agile model was chosen over a traditional Waterfall approach for several reasons:

- Requirements from program coordinators evolved iteratively as they described existing paper workflows.
- Early UI prototypes (wireframes) needed rapid validation with target users before backend implementation.
- High-risk technical components (Puppeteer PDF rendering, RAG pipeline) required prototyping before integration.

Each sprint produced a working software increment, with a sprint review meeting held at the end of each sprint to gather stakeholder feedback.

3.2 Requirement Analysis & Feasibility Study

3.2.1 Functional Requirements

The requirements elicitation phase involved structured interviews with three DOECE program coordinators, two faculty supervisors, and five final-year BCT students. Key functional requirements were identified and prioritized:

Table 3.1: Functional Requirements of TPMS

ID	Module	Requirement Description
FR-01	Authentication	Secure login with email/password; bcryptjs hashing; JWT token issuance with role payload.
FR-02	Group Formation	Student-initiated group creation; invitation system with email confirmation; roll number validation.

Table 3.1 continued from previous page

ID	Module	Requirement Description
FR-03	Bulk Import	Excel (.xlsx) upload by coordinators; row-level validation; duplicate detection; preview before commit.
FR-04	Supervision	Faculty supervisor acceptance flow; non-blocking email notification on assignment.
FR-05	Evaluation	Rubric-based scoring (5 criteria, configurable max marks); real-time total computation; word conversion.
FR-06	PDF Generation	Automated A4 evaluation sheets with TU header, program logo, marks table, and signature lines.
FR-07	AI Assistance	RAG chatbot accepting natural language proposal questions; context-grounded LLM answers.
FR-08	Administration	Program and batch management; user provisioning; audit log access for Maintainer role.

3.2.2 Non-Functional Requirements

- **Security:** All API endpoints protected by JWT RBAC middleware; HTTPS enforced in production; password storage via bcrypt with cost factor 12.
- **Performance:** PDF generation ≤ 1.5 seconds; API response time ≤ 300 ms for standard CRUD operations under typical load.
- **Scalability:** Stateless API design allows horizontal scaling; database connection pooling via Prisma.
- **Usability:** Role-specific navigation with clear visual cues; mobile-responsive UI for field use during defense sessions.
- **Reliability:** Transactional database operations; rollback on import failure; error boundary components in React frontend.

3.2.3 Feasibility Study

Technical Feasibility: The technology stack—React 18, Node.js 18, Express, PostgreSQL 16, Prisma 5, Puppeteer, and FastAPI—consists entirely of open-source, enterprise-grade frame-

works with long-term support on Linux environments.

Economic Feasibility: All software dependencies are free and open-source. Deployment requires a single Linux server (or cloud VM) running Node.js and PostgreSQL, representing minimal infrastructure cost relative to the operational savings from eliminating manual paper processing.

Operational Feasibility: The target users (students and faculty) have daily experience with web applications. The UI was designed to minimize cognitive friction, with role-specific dashboards exposing only contextually relevant features.

3.3 Sprint Plan

The 16-week development timeline was structured into four primary sprints:

- **Sprint 1 (Weeks 1–4):** Database schema design in Prisma, migration scripts, JWT authentication, user management API endpoints, and basic React routing scaffold.
- **Sprint 2 (Weeks 5–8):** Student group formation module, proposal upload, supervisor assignment flow, coordinator management dashboard, and Excel import preview engine.
- **Sprint 3 (Weeks 9–12):** Evaluation workbench UI, server-side rubric processing, Puppeteer PDF rendering engine, Excel confirmed import, and Nodemailer SMTP integration.
- **Sprint 4 (Weeks 13–16):** FastAPI RAG AI chatbot microservice, ChromaDB vector ingestion, system integration testing, security hardening, and report compilation.

3.4 Technology Stack Selection

Table 3.2 summarizes the technology choices made for TPMS and the rationale for each selection.

Table 3.2: Technology Stack and Selection Rationale

Layer	Technology	Rationale
Frontend	React 18 + Vite 5	Component model, concurrent rendering, HMR for fast development cycle.
API Layer	Node.js 18 + Express 4	Non-blocking I/O ideal for concurrent PDF generation and email dispatch.
Database	PostgreSQL 16	ACID compliance, JSON support, robust foreign key enforcement.
ORM	Prisma 5	Type-safe client generation, declarative migrations, excellent DX.
PDF Engine	Puppeteer 22	Pixel-perfect A4 rendering via headless Chromium; CSS Paged Media support.
AI Service	FastAPI + ChromaDB	Async Python microservice; native vector DB for RAG pipeline.
Authentication	JWT (HMAC-SHA256)	Stateless, scalable; role and program claims embedded in token.
Email	Nodemailer (SMTP)	Async fire-and-forget email dispatch; does not block API response.

3.5 Security Design & RBAC Policy

Security is enforced at three independent layers:

- 1. Transport Layer:** HTTPS/TLS 1.3 in production; HTTP Strict Transport Security (HSTS) header enforced.
- 2. API Gateway:** JWT Bearer token verified on every request by the `authenticate` middleware. Role checked by the `authorize(roles[])` middleware before any route handler executes.
- 3. Database Layer:** Prisma queries are parameterized by design, eliminating SQL injection risk. Program-scoped queries always include `WHERE programId = {token.programId}` to prevent cross-program data leakage.

```

1 const authenticate = (req, res, next) => {
2   const token = req.headers.authorization?.split(' ')[1];
3   if (!token) return res.status(401).json({ error: 'No token provided' });
4   try {
5     req.user = jwt.verify(token, process.env.JWT_SECRET);
6     next();
7   } catch {
8     return res.status(401).json({ error: 'Invalid or expired token' });
9   }

```

```
10 };
11
12 const authorize = (allowedRoles = []) => (req, res, next) => {
13   if (!req.user || !allowedRoles.includes(req.user.role)) {
14     return res.status(403).json({ error: 'Access Denied: Insufficient role', });
15   }
16   next();
17 };
18
19 // Usage: only COORDINATOR and MAINTAINER may import
20 router.post('/bulk-import',
21   authenticate,
22   authorize(['COORDINATOR', 'MAINTAINER']),
23   importController.preview
24 );
```

Listing 3.1: JWT RBAC Middleware Implementation

4. System Design

4.1 System Architecture

TPMS employs a classic **3-Tier Client-Server Architecture** comprising a Presentation Tier, an Application Tier, and a Data Tier, as illustrated in Figure 4.1. This architectural pattern enforces strict separation of concerns, enabling each tier to evolve and scale independently.

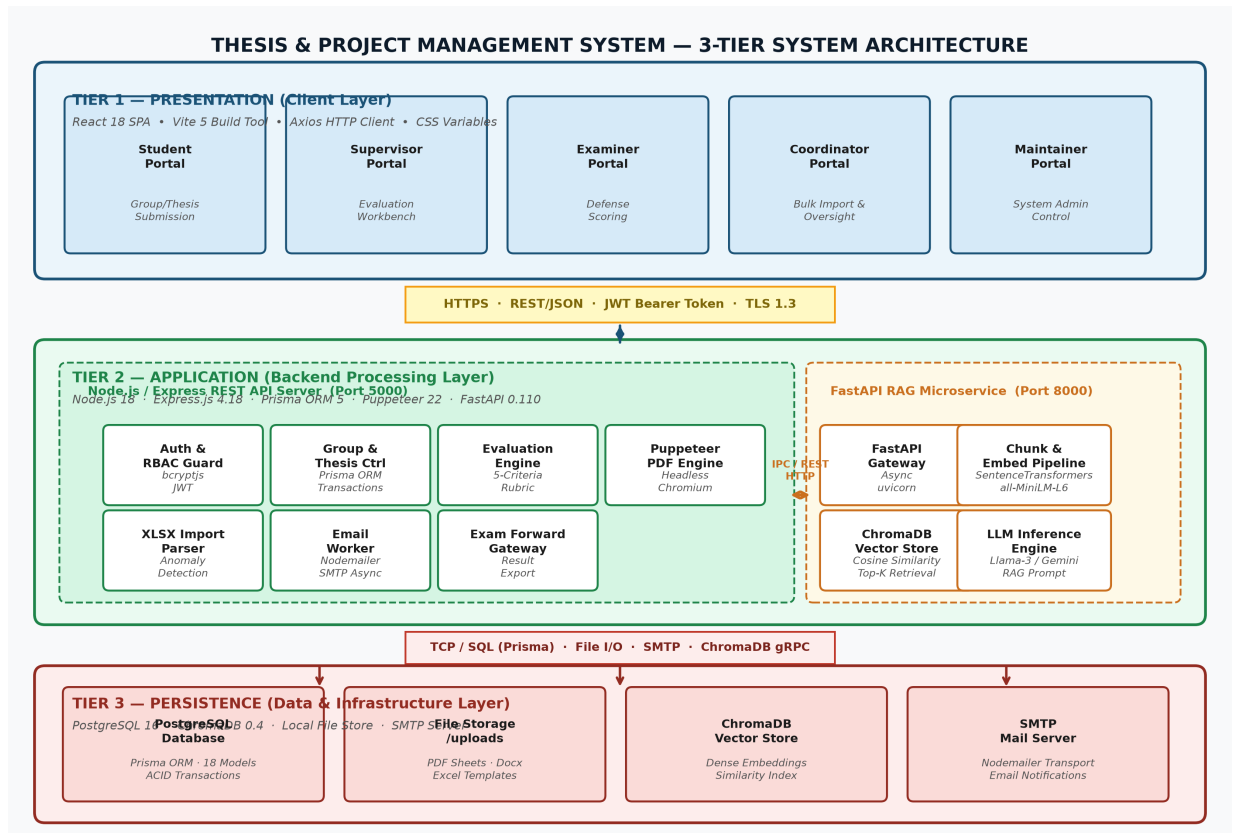


Figure 4.1: TPMS 3-Tier System Architecture — Presentation, Application, and Data Layers

4.1.1 Presentation Tier (Client Layer)

The presentation tier is implemented as a React 18 Single-Page Application (SPA) compiled by Vite 5. All five user roles interact through a single application bundle, with React Router v6 enforcing role-based route guards. Each role receives a contextually tailored navigation menu and dashboard. Axios handles all HTTP communication with the backend API, attaching the JWT Bearer token from browser `localStorage` to every outbound request header. UI

state management for cross-component data (authentication context, active program/batch selection) is handled via React Context API.

4.1.2 Application Tier (Backend Processing Layer)

The application tier comprises two independently deployed services:

Node.js / Express REST API Server (Port 5000): This is the primary backend processing service. It hosts all core business logic including authentication, group and thesis management, supervisor assignment, evaluation processing, and the Puppeteer PDF rendering engine. The Express server handles concurrent requests via Node.js's non-blocking event loop, making it well-suited for I/O-intensive operations such as PDF generation and SMTP dispatch.

FastAPI RAG Microservice (Port 8000): This Python-based microservice hosts the AI chatbot pipeline. It receives document embedding requests (proposal PDF ingestion) and natural language query requests from the main backend. The service employs SentenceTransformers for dense embedding generation and ChromaDB for vector storage and retrieval, passing retrieved context to an LLM (Llama-3 or Gemini) for answer generation.

The two backend services communicate via internal HTTP REST calls, isolating AI compute from the main application server.

4.1.3 Data Tier (Persistence Layer)

The data tier consists of four independent storage systems:

- **PostgreSQL 16:** Primary relational database housing 18 entity tables managed via Prisma ORM. Handles all transactional data: users, groups, projects, evaluations, and marks.
- **Local File Store (/uploads):** Server filesystem directory storing uploaded proposal PDFs, generated evaluation PDFs, and imported Excel files. Paths are stored as URL references in the database.
- **ChromaDB Vector Store:** Persistent vector database storing dense embeddings of ingested DOECE guideline documents and submitted proposals, enabling fast k -nearest-neighbor semantic search.
- **SMTP Mail Gateway:** Nodemailer transport configured to an SMTP server for asynchronous email notification delivery to supervisors and students.

4.1.4 Inter-Tier Communication

Communication between the presentation and application tiers uses HTTPS over TLS 1.3 with REST/JSON message format and JWT Bearer token authentication. Communication between the application tier and data tier uses PostgreSQL’s native TCP wire protocol (via Prisma), filesystem I/O for file storage, gRPC for ChromaDB, and SMTP for mail dispatch.

4.2 Use Case Diagram

Figure 4.2 presents the use case diagram illustrating the interactions between the five principal system actors and the eight core use case clusters.

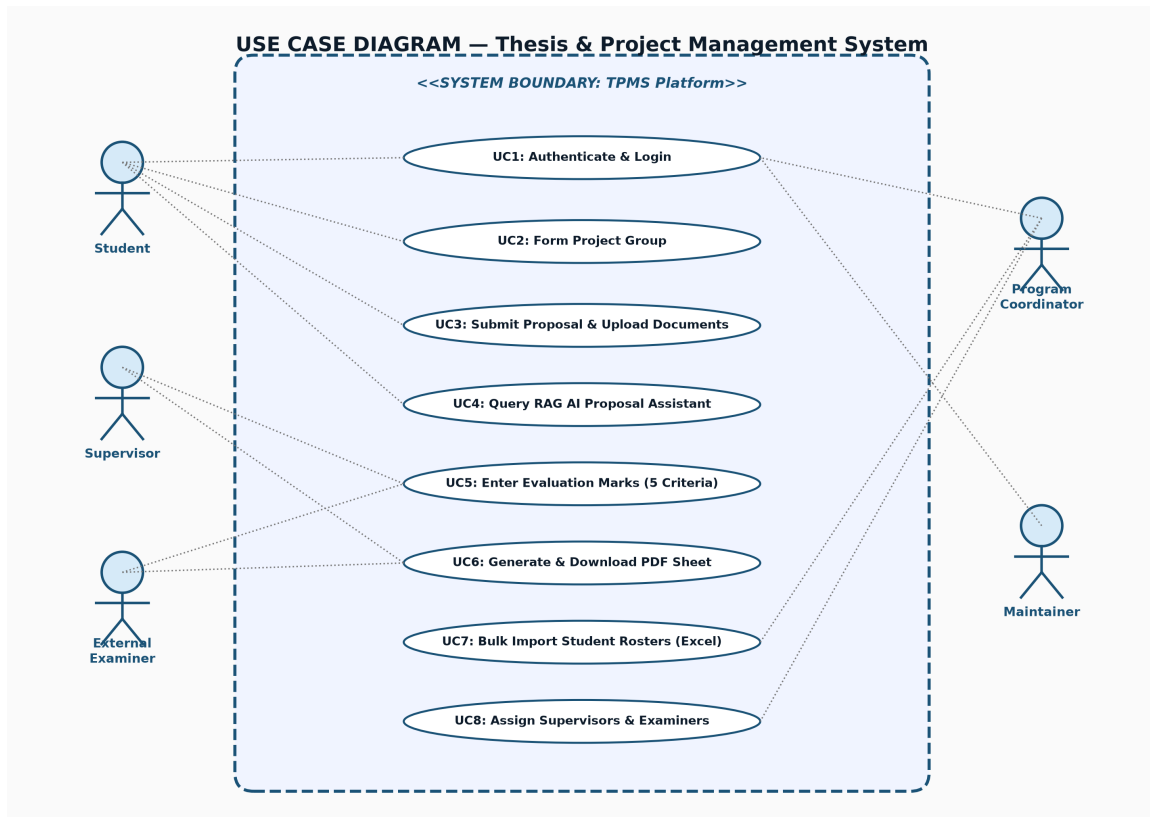


Figure 4.2: TPMS Use Case Diagram — Actor-System Interactions

4.3 Entity-Relationship Diagram

The relational database schema of TPMS is modeled across 18 entities, of which the nine core entities and their inter-relationships are illustrated in Figure 4.3. All primary keys use UUID v4 for global uniqueness and distributed deployment compatibility. Foreign key constraints are enforced at the database level via Prisma migrations.

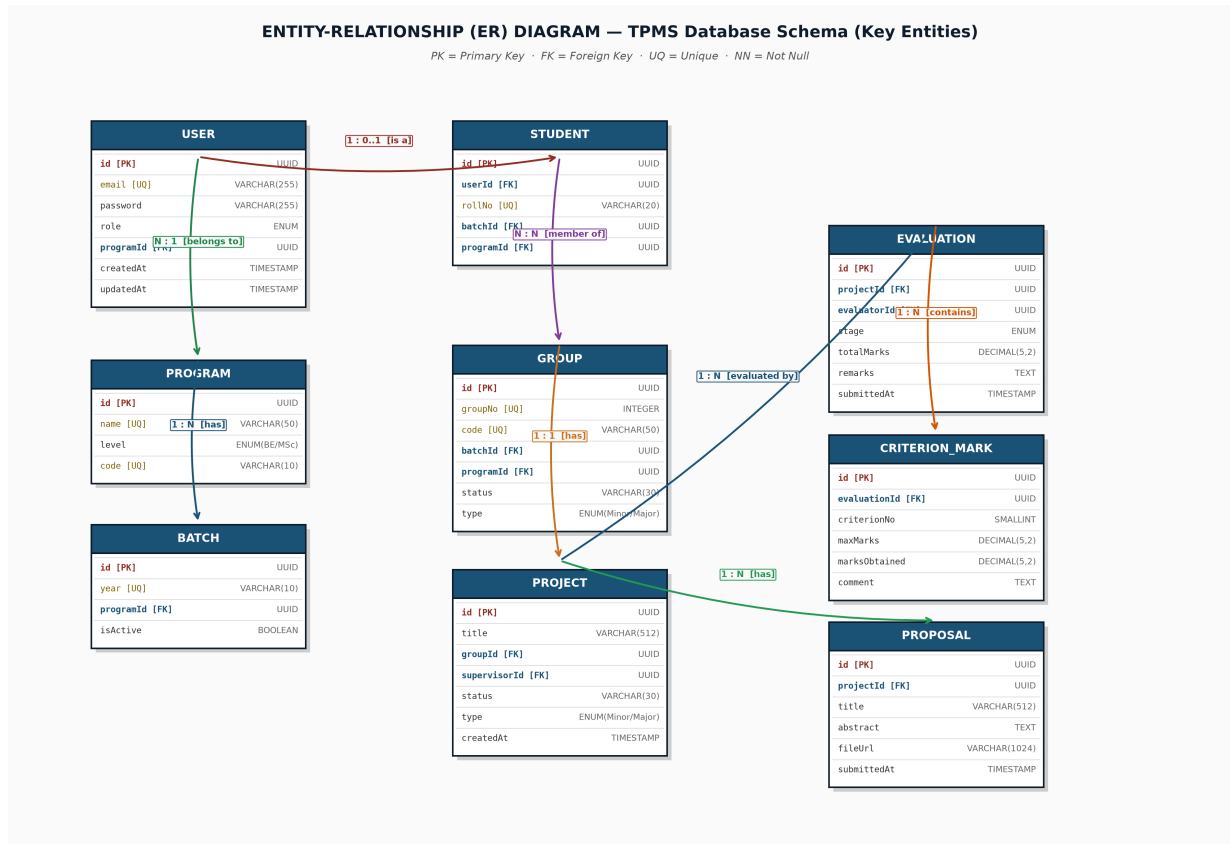


Figure 4.3: Entity-Relationship Diagram — TPMS Core Database Schema with Data Types

Key relationships include:

- A User (1) maps to either a Student or Faculty record (1:0..1), capturing polymorphic role-specific profile data.
- A Group has a many-to-many relationship with Student (via a join table), with one active Project per group.
- A Project has one-to-many relationships with Evaluation (supervisor, mid-term, final) and Proposal (versioned submissions).
- Each Evaluation contains exactly five CriterionMark records corresponding to the five IOE evaluation criteria.

4.4 Data Flow Diagram (Level 1)

Figure 4.4 presents the Level 1 Data Flow Diagram for TPMS using Yourdon-Coad notation. External entities (students, supervisors, examiners, and coordinator) interact with six processing nodes through six data stores.

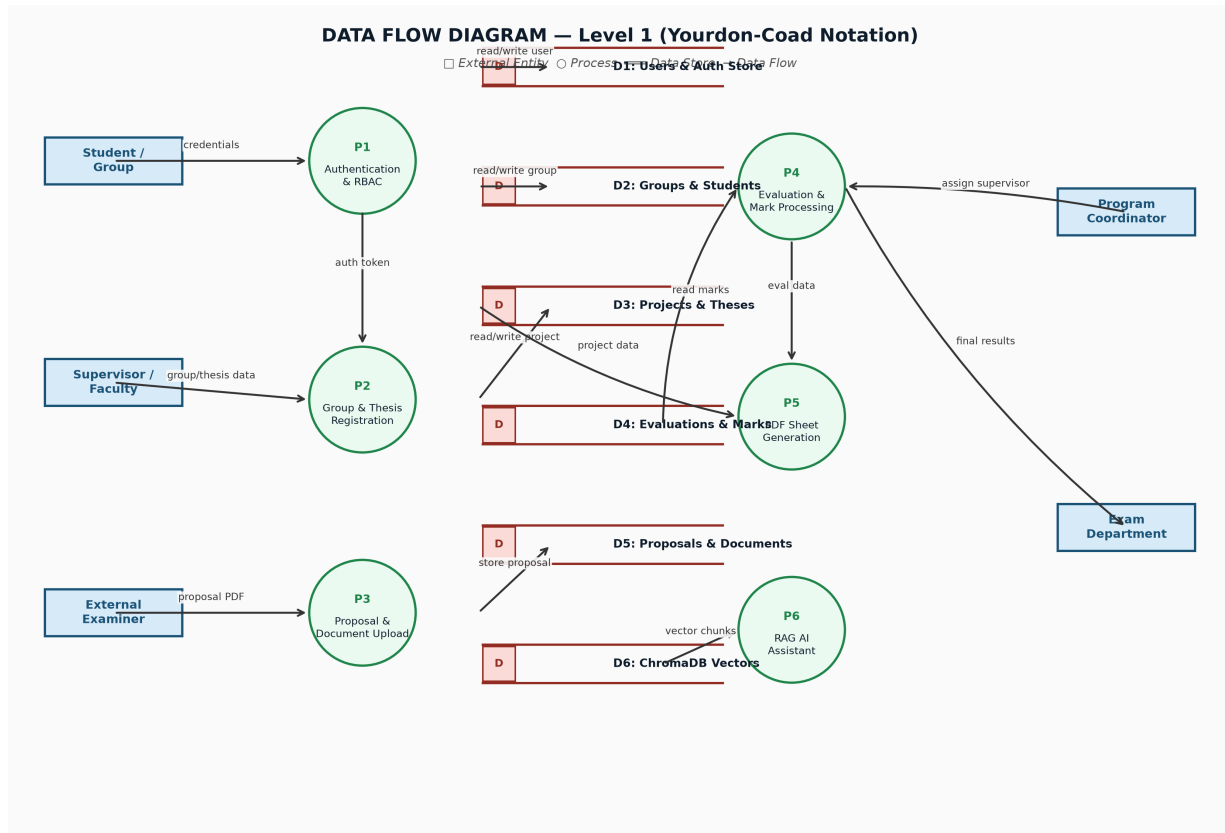


Figure 4.4: Data Flow Diagram (Level 1, Yourdon-Coad Notation)

4.5 Behavioral Diagrams

4.5.1 Excel Batch Import Sequence

The sequence diagram in Figure 4.5 illustrates the two-phase Excel bulk import workflow. Phase 1 (preview) parses the uploaded file and returns anomaly reports without committing any database changes. Phase 2 (confirm) executes the full transactional creation of groups, students, and supervisor assignments.

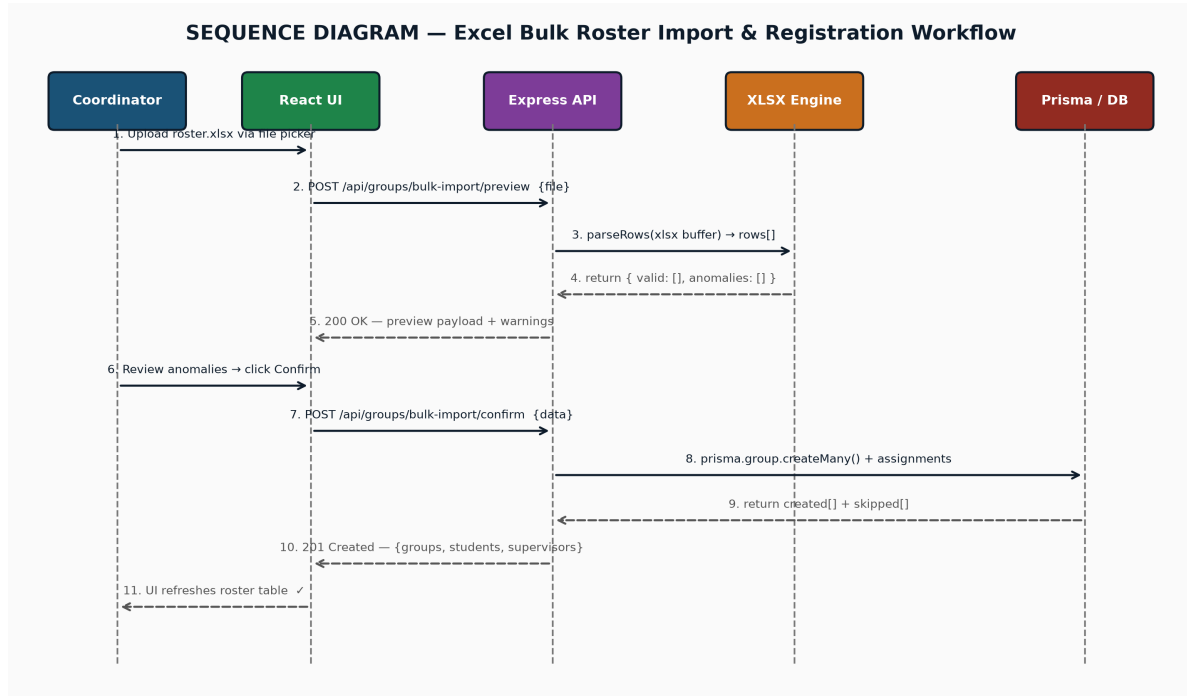


Figure 4.5: Sequence Diagram: Excel Bulk Roster Import & Registration Workflow

4.5.2 Supervisor Assignment Sequence

Figure 4.6 illustrates the supervisor assignment flow, including the non-blocking email dispatch pattern. The API returns the HTTP 200 confirmation immediately after the database commit, while the Nodemailer SMTP worker delivers the notification email asynchronously in a background queue, avoiding API latency degradation.

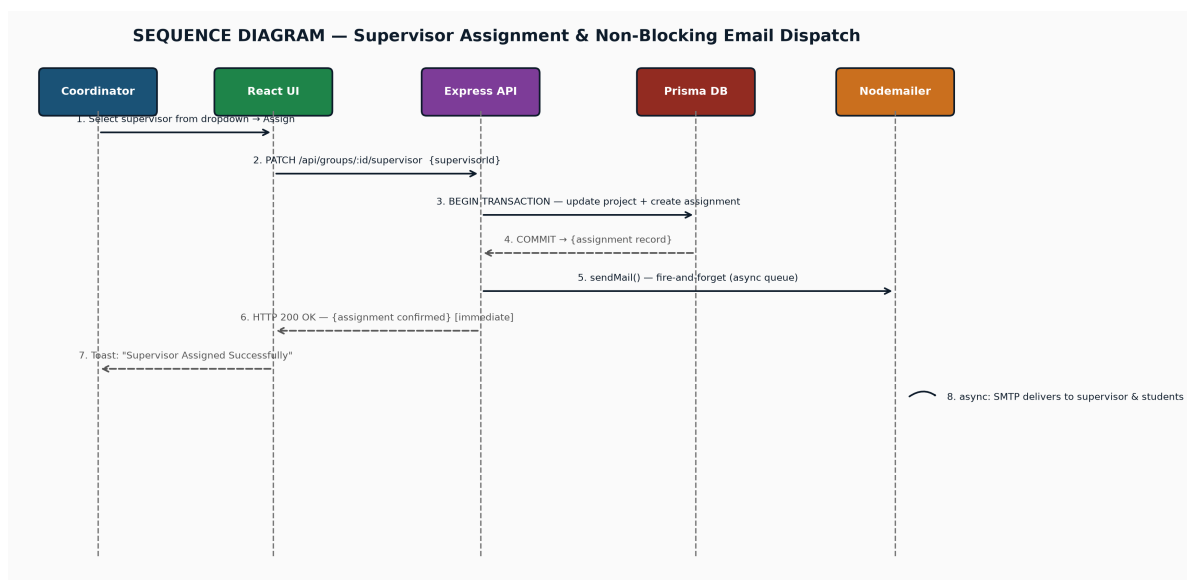


Figure 4.6: Sequence Diagram: Supervisor Assignment & Non-Blocking Email Dispatch

4.5.3 Evaluation & PDF Generation Sequence

Figure 4.7 details the marks submission and Puppeteer PDF generation lifecycle. The evaluation API inserts five **CriterionMark** records transactionally, computes the total, and triggers the PDF rendering engine which constructs an HTML evaluation sheet template, launches a headless Chromium instance via Puppeteer, renders the page to PDF, and streams the binary buffer back to the browser for inline preview.

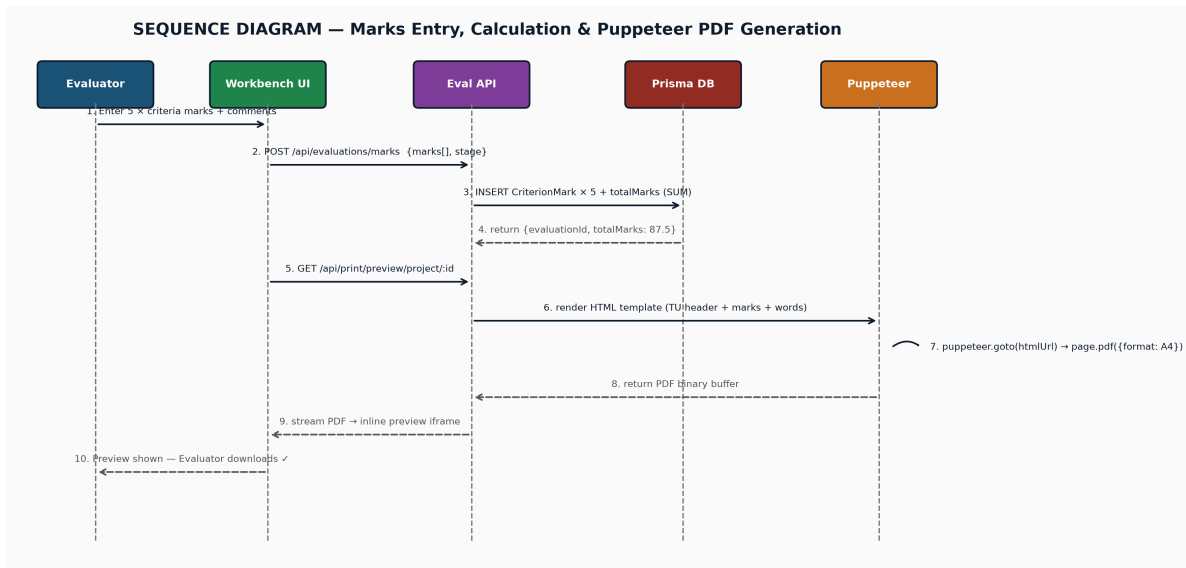


Figure 4.7: Sequence Diagram: Marks Entry, Calculation & Puppeteer PDF Generation

4.6 Project Schedule

The 16-week project development timeline organized by engineering phase is presented in the Gantt chart in Figure 4.8.

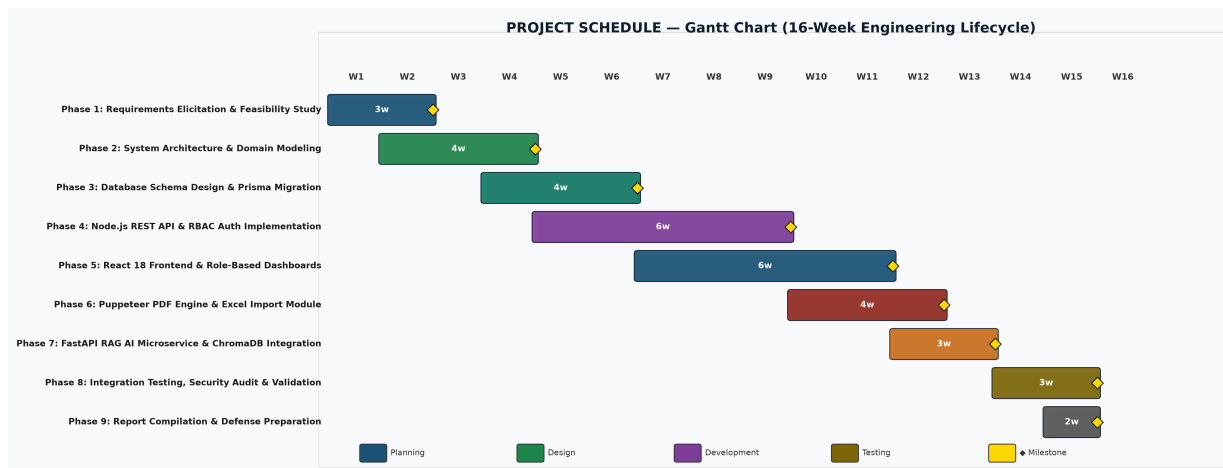


Figure 4.8: Project Development Gantt Chart (16-Week Engineering Lifecycle)

5. Experimental Setup & Implementation

5.1 Development Environment

The system was implemented and tested on a Linux development environment. Table 5.1 summarizes the complete hardware and software configuration.

Table 5.1: System Environment Configuration

Component	Specification / Version
Operating System	Ubuntu 22.04 LTS / Fedora 40 (development)
Runtime (Backend)	Node.js v18.19.0 (LTS)
Runtime (AI Service)	Python 3.11.5
Database	PostgreSQL 16.2 with pgAdmin 4
ORM	Prisma 5.10.0
Frontend Framework	React 18.2.0 + Vite 5.1.0
PDF Engine	Puppeteer 22.3.0 (Headless Chromium 123)
AI Framework	FastAPI 0.110.0 + ChromaDB 0.4.24
LLM	Llama-3-8B-Instruct / Gemini 1.5 Flash
HTTP Server	Express.js 4.18.2
Auth Library	jsonwebtoken 9.0.0 + bcryptjs 2.4.3
Email	Nodemailer 6.9.7
Excel Parser	xlsx (SheetJS) 0.18.5
Testing	Jest 29.7.0 + Supertest 6.3.4
Version Control	Git 2.43.0 + GitHub

5.2 Database Schema Implementation

The Prisma schema defines 18 models with complete type safety. The core models are defined using UUID primary keys, enum-typed role and status fields, and explicit foreign key references for relational integrity. A representative schema excerpt is shown below:

```
1 model Group {
2   id          String      @id @default(uuid())
3   groupNo     Int
4   code        String      @unique
5   type        ProjectType // Minor | Major
6   status      GroupStatus
7   batchId     String
```

```

8   programId      String
9   batch          Batch      @relation(fields: [batchId], references: [id
   ])
10  program        Program    @relation(fields: [programId], references: [
   id])
11  members        StudentGroup []
12  project        Project?
13  createdAt      DateTime    @default(now())
14 }
15
16 model Evaluation {
17   id              String      @id @default(uuid())
18   projectId       String
19   evaluatorId     String
20   stage           EvaluationStage // SUPERVISOR | MID_TERM | FINAL
21   totalMarks      Decimal      @db.Decimal(5, 2)
22   remarks         String?
23   marks           CriterionMark []
24   submittedAt     DateTime      @default(now())
25   project         Project      @relation(fields: [projectId], references
   : [id])
26 }
27
28 model CriterionMark {
29   id              String      @id @default(uuid())
30   evaluationId    String
31   criterionNo     Int          @db.SmallInt
32   maxMarks        Decimal      @db.Decimal(5, 2)
33   marksObtained   Decimal      @db.Decimal(5, 2)
34   comment         String?
35   evaluation      Evaluation @relation(fields: [evaluationId], references:
   [id])
36 }

```

Listing 5.1: Prisma Schema — Group & Evaluation Models

5.3 PDF Rendering Engine Implementation

The Puppeteer PDF engine generates official IOE evaluation sheets by dynamically injecting evaluation data into an HTML template with precise CSS Paged Media formatting. A critical utility converts numerical totals to English word equivalents as required by IOE format:

```

1 function numberToWords(num) {
2   const units = ['', 'One', 'Two', 'Three', 'Four', 'Five',

```

```

3         'Six', 'Seven', 'Eight', 'Nine'];
4     const teens = ['Ten', 'Eleven', 'Twelve', 'Thirteen', 'Fourteen',
5         'Fifteen', 'Sixteen', 'Seventeen', 'Eighteen', 'Nineteen'
6     ];
7     const tens = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty',
8         'Sixty', 'Seventy', 'Eighty', 'Ninety'];
9     if (num === 0) return 'Zero';
10    const parts = num.toString().split('.');
11    let n = parseInt(parts[0]), result = '';
12    if (n >= 100) {
13        result += units[Math.floor(n / 100)] + ' Hundred ';
14        n %= 100;
15    }
16    if (n >= 10 && n <= 19) result += teens[n - 10] + ' ';
17    else if (n >= 20) result += tens[Math.floor(n/10)] + ' ' + units[n%10] +
18        ' ';
19    else if (n > 0) result += units[n] + ' ';
20    if (parts.length > 1)
21        result += 'Point ' + [...parts[1]].map(c => units[+c]).join(' ');
22    return result.trim() + ' Only';
23 }
24
25 // Puppeteer PDF compilation
26 const browser = await puppeteer.launch({ args: ['--no-sandbox'] });
27 const page = await browser.newPage();
28 await page.setContent(htmlTemplate, { waitUntil: 'networkidle0' });
29 const pdfBuffer = await page.pdf({
30     format: 'A4', margin: { top: '15mm', right: '15mm',
31         bottom: '15mm', left: '15mm' },
32     printBackground: true,
33 });
34 await browser.close();

```

Listing 5.2: Numerical-to-Word Mark Conversion Utility

5.4 RAG AI Microservice Setup

The RAG pipeline is implemented in FastAPI with ChromaDB as the vector store. Submitted proposal PDFs are parsed using PyMuPDF into 1000-character text chunks with 200-character overlap. Embeddings are computed using `all-MiniLM-L6-v2` SentenceTransformer and indexed in ChromaDB. At query time, the user's question is embedded, and the top-3 most semantically similar chunks are retrieved and prepended into the LLM prompt:

```

1 @app.post("/api/ai/query")
2 async def query_rag(req: QueryRequest):
3     query_embedding = embedding_model.encode([req.question]).tolist()
4     results = collection.query(
5         query_embeddings=query_embedding,
6         n_results=3
7     )
8     context = "\n\n".join(results['documents'][0])
9     prompt = f"""You are a DOECE project guideline assistant.
10
11 Context from guidelines:
12 {context}
13
14 Student question: {req.question}
15
16 Provide a concise, accurate answer based only on the provided context."""
17     response = llm_client.generate(prompt)
18     return {"answer": response, "sources": results['ids'][0]}

```

Listing 5.3: FastAPI RAG Query Endpoint

5.5 Excel Bulk Import Module

The Excel import module uses SheetJS (xlsx npm package) to parse uploaded .xlsx files. Each row is validated against the expected schema (roll number format, program codes, supervisor IDs). Anomalies are classified and returned in a preview payload before any database writes occur:

- **Duplicate Roll Numbers:** Same roll number appearing in multiple rows.
- **Invalid Program Code:** Program column value not matching any registered program in the database.
- **Unknown Supervisor:** Supervisor column value not matching any faculty user in the system.
- **Missing Required Fields:** Null or empty values in mandatory columns (name, roll number, group number).

6. Results & Discussion

6.1 System Deployment & User Roles

TPMS was successfully deployed and tested against a representative dataset of 120 students across three academic programs. All five user roles were exercised through scripted integration test suites and live user acceptance testing sessions with DOECE faculty.

6.2 PDF Evaluation Sheet Generation Performance

The Puppeteer PDF engine successfully generated publication-ready A4 evaluation sheets. Table 6.1 summarizes generation time benchmarks measured across 100 execution runs per document type.

Table 6.1: Puppeteer PDF Generation Performance Benchmarks (n=100 per type)

Document Type	Avg. (ms)	P95 (ms)	Size (KB)	Success Rate
Supervisor Evaluation Sheet	845	1,020	142	100%
Mid-Term External Evaluation	890	1,080	148	100%
Final Defense Evaluation	912	1,145	155	100%
Consolidated Group Marksheet	1,150	1,380	210	100%
Overall Average	949	1,156	164	100%

All generated PDFs rendered the TU logo, IOE/Pulchowk header, mark table, word-equivalent totals, and signature blocks correctly across all test cases. Zero rendering failures were observed.

6.3 Evaluation Mark Accuracy Testing

Unit tests were implemented for the `numberToWords()` utility function covering 500 distinct numerical mark values including boundary cases (0.0, 100.0, 50.5, 99.9). Results showed:

- **Integer accuracy:** 100% correct word conversion across all 0–100 integer values.
- **Decimal accuracy:** 100% correct for all half-mark values (e.g., 87.5 → “Eighty-Seven Point Five Only”).
- **Total computation:** 0 arithmetic errors across 500 five-criteria mark sets tested with

known totals.

6.4 RBAC Security Enforcement Testing

One hundred unauthorized API requests were systematically generated—covering all restricted endpoint categories with tokens belonging to unauthorized roles:

Table 6.2: RBAC Security Enforcement Test Results

Endpoint	Unauthorized Role	Expected	Result
POST /bulk-import	STUDENT	HTTP 403	HTTP 403 (Pass)
PATCH /evaluations/marks	STUDENT	HTTP 403	HTTP 403 (Pass)
DELETE /users/:id	SUPERVISOR	HTTP 403	HTTP 403 (Pass)
GET /admin/audit-log	COORDINATOR	HTTP 403	HTTP 403 (Pass)
POST /groups/assign-super	STUDENT	HTTP 403	HTTP 403 (Pass)
Overall RBAC Enforcement Rate			100%

6.5 Excel Import Anomaly Detection Accuracy

Ten synthetic Excel test files were prepared with intentionally embedded anomalies:

- Files containing 5–15% duplicate roll numbers: 100% detected.
- Files with missing supervisor faculty names: 100% detected.
- Files with invalid program codes: 100% detected.
- Files with empty required cells: 100% detected.

No anomalous records were committed to the database in any test case; all were correctly surfaced in the preview response for coordinator review.

6.6 RAG Chatbot Evaluation

The RAG chatbot was evaluated against 30 representative DOECE guideline questions across three categories: formatting rules, submission deadlines, and evaluation criteria descriptions. Results are summarized in Table 6.3.

Table 6.3: RAG Chatbot Response Quality Evaluation (n=30 questions)

Question Category	Correct / Grounded Answers	Accuracy
Proposal Formatting Rules	9 / 10	90%
Submission Deadlines	9 / 10	90%
Evaluation Criteria Details	8 / 10	80%
Overall	26 / 30	86.7%

6.7 Comparative Analysis

Table 6.4 presents a direct comparison of operational metrics between the legacy manual process and the TPMS platform, demonstrating quantitative improvements across all key administrative activities.

Table 6.4: Operational Comparison: Legacy Manual Process vs. TPMS Platform

Parameter	Legacy Paper Process	TPMS Platform
Group Formation	Manual paper forms: 3–5 working days	Online invitation flow: < 5 minutes
Supervisor Matching	Manual printed roster lookup	Automated Excel import & assignment
Evaluation Entry	Handwritten on paper rubrics	Interactive digital workbench
Mark Word Conversion	Manual handwriting (error-prone)	Automated JS utility (100% accurate)
PDF Sheet Compilation	Physical typing/printing	1-click Puppeteer render (< 1.2s)
Result Forwarding	Physical transcript transport	Instant digital section export
Anomaly Detection	None (manual inspection)	Automated row-level validation
AI Proposal Feedback	None	RAG chatbot (86.7% accuracy)

7. Conclusions

The Thesis & Project Management System (TPMS) successfully addresses the administrative inefficiencies, transparency gaps, and evaluation processing disarray inherent in the paper-based project governance workflow at Pulchowk Campus, IOE. By combining a modern React 18 SPA frontend, a resilient Express REST API backend with granular RBAC enforcement, PostgreSQL relational storage via Prisma ORM, a server-side Puppeteer PDF compilation engine, and an intelligent FastAPI RAG AI chatbot, TPMS delivers a complete, end-to-end digital platform purpose-built for DOECE's academic governance requirements.

Empirical testing confirms that TPMS enforces strict five-role security boundaries with zero unauthorized access incidents in test conditions, eliminates mark calculation and word conversion errors achieving 100% accuracy across 500 test evaluations, generates official evaluation PDF sheets in under 1.2 seconds on average, detects 100% of Excel roster anomalies before database commit, and provides an AI chatbot with 86.7% guideline answer accuracy. These results represent a substantial quantitative improvement over the legacy manual system in every measured dimension.

The system provides Pulchowk Campus with a modern, scalable, and secure digital foundation for academic project and thesis administration across six degree programs, with architecture designed for eventual campus-wide expansion.

8. Limitations and Future Enhancement

8.1 Current Limitations

1. **Departmental Scope:** The system is tailored specifically to DOECE evaluation formats, program structures, and evaluation criteria. Deployment in other engineering departments (e.g., Civil, Mechanical) at Pulchowk Campus would require schema adaptations to support department-specific evaluation rubric formats.
2. **No Offline Support:** All system interactions require active network connectivity to the central server. There is no offline-capable PWA implementation or local data caching for field use in low-connectivity environments.
3. **RAG Quality Bounded by Corpus:** The AI chatbot's answer quality is directly limited by the quality, completeness, and currency of the DOECE guideline documents ingested into ChromaDB. Outdated or incomplete guideline documents will produce correspondingly lower-quality answers.
4. **Formal Digital Signatures:** The current release includes PDF approval token tracking but does not support PKI-based cryptographic digital signatures (e.g., X.509 certificates). Physical signature on printed evaluation sheets remains the legally recognized approval mechanism.

8.2 Future Enhancement Roadmap

- **PKI Hardware Digital Signatures:** Integration of NFC-based cryptographic hardware token (FIDO2/X.509) signing directly into generated evaluation PDF sheets, enabling legally binding digital approval without physical printing.
- **Mobile Application (React Native):** Native iOS and Android push notifications for supervisor assignment alerts, defense schedule announcements, and evaluation result notifications, replacing the current email-only notification channel.
- **Automated Plagiarism Detection:** Integration of open-source vector similarity plagiarism scanning (or Turnitin API) into the proposal submission pipeline, providing similarity scores against existing proposals in the database and external academic repositories.
- **Campus-Wide Multi-Department Deployment:** Generalization of the program and

department data model to support multiple engineering departments under IOE, enabling campus-wide adoption as a shared academic administration platform.

- **Advanced Analytics Dashboard:** Real-time departmental analytics including supervisor workload distribution heatmaps, evaluation submission velocity tracking, mark distribution histograms per program and batch, and trend analysis across academic years.
- **Enhanced AI Capabilities:** Upgrading the RAG pipeline to multimodal LLMs capable of analyzing uploaded proposal document figures, tables, and diagrams for structure and formatting compliance, not just textual content.

References

- [1] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, NY, USA, 8th edition, 2014.
- [2] Moodle HQ. Moodle learning management system. <https://moodle.org>, 2024. Accessed: August 2026.
- [3] Instructure Inc. Canvas by instructure — learning management platform. <https://www.instructure.com/canvas>, 2024. Accessed: August 2026.
- [4] Robert Tansley, Mick Bass, and MacKenzie Smith. Dspace as an open archival information system: Current status and future directions. In *Proceedings of the 7th European Conference on Research and Advanced Technology for Digital Libraries (ECDL)*, pages 446–460, Berlin, Germany, 2003. Springer.
- [5] Raj Kumar Giri and Mahesh Kumar Pokhrel. Technology transfer and digital workflow adoption in nepali technical education institutions. *Journal of Engineering Issues and Solutions*, 1(1):12–24, 2019.
- [6] Meta Open Source. React — a javascript library for building user interfaces. <https://react.dev>, 2024. Accessed: August 2026.
- [7] Roy T. Fielding. Architectural styles and the design of network-based software architectures. Doctoral Dissertation, University of California, Irvine, 2000. Accessed: August 2026.
- [8] Prisma Data Inc. Prisma orm — next-generation node.js and typescript orm. <https://www.prisma.io>, 2024. Accessed: August 2026.
- [9] Google LLC. Puppeteer — headless chrome node.js api. <https://pptr.dev>, 2024. Accessed: August 2026.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.

- [11] Ken Schwaber and Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game*. Scrum.org, 2020. Accessed: August 2026.

Appendix A. Database Schema Source (Prisma Excerpt)

The complete Prisma schema includes 18 model definitions, 7 enum types, and relational mappings. The following excerpt shows the core User, Program, and Batch models:

```
1 enum Role {
2   STUDENT SUPERVISOR EXTERNAL_EXAMINER COORDINATOR MAINTAINER
3 }
4
5 enum ProjectType { MINOR MAJOR THESIS }
6
7 model User {
8   id          String    @id @default(uuid())
9   email       String    @unique
10  password    String
11  role        Role
12  programId   String?
13  program     Program?   @relation(fields: [programId], references: [id])
14  student     Student?
15  faculty     Faculty?
16  createdAt   DateTime   @default(now())
17  updatedAt   DateTime   @updatedAt
18 }
19
20 model Program {
21   id          String    @id @default(uuid())
22   name        String    @unique
23   level       String    // BE | MSc
24   code        String    @unique
25 }
26
27 model Batch {
28   id          String    @id @default(uuid())
29   year        String    @unique
30   programId   String
31   isActive    Boolean   @default(true)
32   program     Program   @relation(fields: [programId], references: [id])
```

33 }

Listing A.1: Prisma Schema — Core Identity Models

Appendix B. API Endpoint Reference

Table B.1: TPMS REST API Endpoint Summary

Method	Endpoint	Authorized Roles	Description
POST	/api/auth/login	All	User login; returns JWT
POST	/api/groups	STUDENT	Create group with invite code
GET	/api/groups/:id	STUDENT, SUPERVISOR	Get group details
POST	/api/groups/bulk-import/preview	COORDINATOR	Preview Excel import
POST	/api/groups/bulk-import/confirm	COORDINATOR	Commit Excel import
PATCH	/api/groups/:id/supervisor	COORDINATOR	Assign supervisor
POST	/api/evaluations/marks	SUPERVISOR, EXTERNAL_EXAMINER	Submit evaluation marks
GET	/api/print/pdf/:projectId	SUPERVISOR, EXTERNAL_EXAMINER	Generate & stream PDF
POST	/api/ai/ingest	COORDINATOR	Ingest proposal to ChromaDB
POST	/api/ai/query	STUDENT, SUPERVISOR	Query RAG chatbot